Multi Agent Pacman Capture the Flag
Course: Modern Game AI, Assignment 2

Dimitrios Kourntidis(s3841863), Myriana Miltiadous(s3699463), Mojhde Hamidi(s4023803)

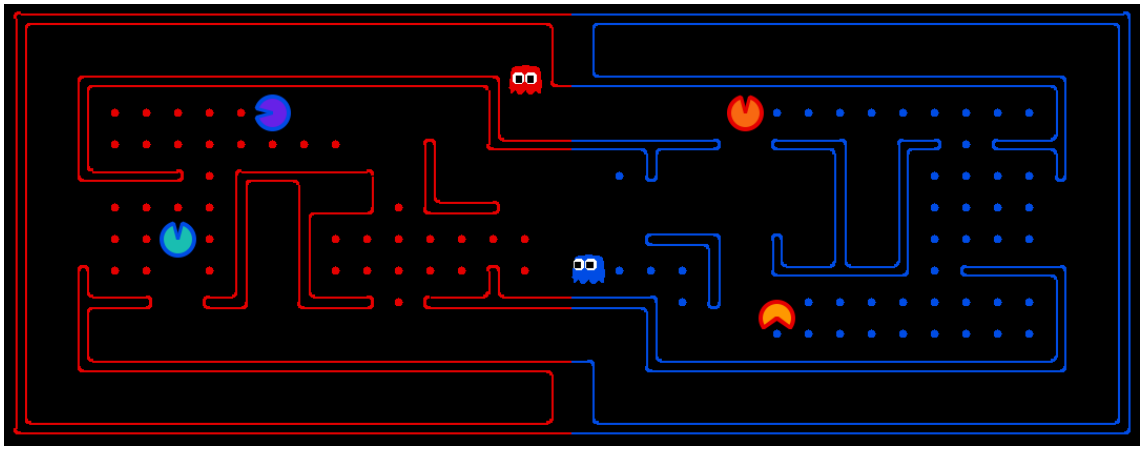

Figure 1: Pacman Capture the Flag maze layout. The image was taken from Pacman Capture the Flag website[1].

1 Introduction

This report presents the implementation of agents to play the Pacman Capture the Flag game [1] as part of a group project. The primary objectives were to develop effective agents using various artificial intelligence techniques.

Pacman Capture the Flag is a multi-agent game where two teams, each controlling Pacman agents and ghost agents, compete to capture food pellets in the opposing team's territory while avoiding being caught by the other team's ghosts. The gameplay requires agents to navigate the maze-like environment, reason about opponent behavior, plan ahead, and coordinate offensive and defensive strategies as a team.

Our group implemented three different agents to tackle this challenge: an MCTS agent, a Q-Learning agent, and a heuristic-based agent. The MCTS agent uses Monte Carlo simulations to construct a search tree and make informed decisions by evaluating future game states. The Q-Learning agent employs reinforcement learning techniques to learn an optimal policy through repeated gameplay experiences. The heuristic-based agent relies on hand-crafted rules and heuristics to guide its decision-making process.

The following sections will explore the details of each agent's implementation, including the algorithms and techniques used. Furthermore, a comparative evaluation of the different agents will be presented, after running a small tournament.

2 Methodology

2.1 MCTS agent

2.1.1 Implementation

Monte Carlo Tree Search (MCTS) is a heuristic search algorithm that has been successfully applied to many challenging domains, including game-playing, planning, and optimization problems. MCTS combines the precision of tree search with the generality of random sampling, making it well suited for exploring large search spaces efficiently.

The key idea behind MCTS is to construct a search tree incrementally by running many randomized simulations (called rollouts or playouts) from the current game state. The results of these simulations are used to guide the tree growth towards more promising regions of the search space. This approach allows the MCTS to selectively explore the most relevant parts of the game tree while avoiding the need to build the entire tree, which quickly becomes intractable in complex domains.

MCTS operates in an iterative manner, with each iteration consisting of four main steps: selection, expansion, simulation, and backpropagation. During the selection, the algorithm traverses the existing tree by selecting the most promising child nodes based on a carefully designed strategy. In the expansion step, one or more new child nodes are added to the tree to represent the new game states to explore. The simulation phase involves playing out the game from new nodes by making random moves until a terminal state is reached. Finally, the result of each simulation was backpropagated through the tree, updating the node statistics to inform future selection decisions

2.1.2 Adjusting Parameters

The MCTS algorithm offers various parameters that can be adjusted to optimize performance in a particular game environment. The following parameters were taken into account: Max Depth, Exploration Constant (epsilon), Feature Weights, and Time Limit.

1. Max Depth: The maximum depth of the search tree, limiting the exploration depth. For implementation, a value of 15 was chosen to balance the computational complexity and search depth.

2. Exploration Constant (epsilon): This parameter controls the balance between exploitation (selecting the most promising node) and exploration (selecting a random node). A value of 0.1 was used.

3. Feature Weights: The algorithm uses a linear combination of features to evaluate the game states. The weights associated with these features were manually tuned based on domain knowledge and empirical observations. For instance, in the offensive agent's feature set, a weight of 100 was assigned to the `get\_food` feature, encouraging the agent to prioritize the collection of food pellets.

4. Time Limit: The time allowed for the MCTS search process to run before returning to a decision. A value of 0.99 seconds was used, providing a reasonable trade-off between search depth and decision-making time.

The parameter values were determined through iterative experimentation and evaluation of the agent's performance in various game scenarios. The tuning process aimed to strike a balance between exploration and exploitation, computational efficiency, and prioritization of game-specific objectives, such as capturing food pellets or defending the team's territory.

It is worth noting that the parameter-tuning process was not entirely automated and involved manual adjustments based on empirical observations and domain knowledge. Future work could explore automated parameter tuning techniques, such as grid search or Bayesian optimization, to further enhance agent performance.

2.2 Q-learning Agent

After the Monte Carlo Tree Search (MCTS) agent was successfully implemented and tested, we turned our attention to creating a new agent that uses reinforcement learning instead of Tree

Search. Motivated by the proven efficiency and lower time complexity of Q-learning approximation methods in multi-agent settings, we started developing an agent that learns via reinforcement learning, utilizing Q-learning approximation in particular [2, 4, 3].

As our agent explores different game states and acts, it learns by modifying its weights. In the beginning of the learning phase, the agent prioritizes discovery. But as the game goes on, exploitation becomes more frequent since players are aware of the rewards. In order to support this learning process, a linear function that approximates game states and generates rewards based on the agent's behaviors is computed using features and weights.
To be more specific, the approximation Q-learning approach works using the following formula:

$$Q(s,a) = \sum_{i=1}^{n} w_i \cdot f_i(s,a)$$

where:

- $Q(s,a)$: Q-value, for taking action a from state s onwards.
- $w_1, w_2, \ldots, w_n$: Weights for features $f_1(s,a), f_2(s,a), \ldots, f_n(s,a)$, determining their importance in approximating the Q-value.
- $f_1(s,a), f_2(s,a), \ldots, f_n(s,a)$: Features of state-action pair (s,a).

For any state-action pair, this formula approximates the Q-value, which is an estimate of the total reward an agent might expect from executing a particular action starting in a given state. As a result, the agent's weights need to be updated in accordance with its training experiences in order for learning to take place effectively. This involves calculating the difference between the current state and the previous state, represented as difference $= [r + \gamma \cdot \max_{a'} Q(s', a')] - Q(s,a)$, where r represents the reward and γ the discount factor, that determines the importance of future rewards compared to immediate rewards. Then the weights are susequently updated, using the formula $w_i = w_i + \alpha \cdot \text{difference} \cdot f_i(s,a)$, where α represents the learning rate.

Although this approach seems straightforward, there is the likelihood that the agent's policy won't converge and that the estimated Q-values for state-action pairings will diverge or rise too much over time, which would cause instability in the learning process. This problem, inherent to approximate Q-learning, can be caused by overestimating Q-values, making incorrect estimates, or conducting insufficient exploration. However, mitigating strategies can be employed, which we have incorporated using specific tools, that are going to be mentioned.

2.2.1 Implementation

For implementing our Q approximation team, we decided to assign one agent to be mostly offensive and the other to be defensive in order to maximise our team's performance in the Pacman Capture the Flag game, where each team consists of two agents that can play both offensive and defensive roles. Even though these agents have different policies and operate autonomously, they cooperate by exchanging information to increase the efficacy of their squad. Once the Q-values and weights of our agents are calculated, the next action is determined based on an exploration rate. Furthermore, a buffer is used to keep values from getting unnecessarily large or tiny while updating weights based on active characteristics of the game state. This buffer stores the last 1000 values and rewards, from which 100 instances are sampled to average out the current Q-value during updates and this value is kept, instead of the actual current q value of equation difference $= [r + \gamma \cdot \max_{a'} Q(s', a')] - Q(s,a)$, mentioned before.
The features that we decided on for our agents are the following:

1. Closest food to the agent
2. Nearest recently eaten food by the opponent
3. Points currently carried by the agent
4. Proximity of ghosts and Pacman to the agent

Our offensive player's primary goal is to score as many points as possible and get them back to the team while avoiding obstacles— ghosts—. It can, however, also strengthen defences. So, we made sure with the use of weights that, the agent is awarded positively for doing actions that lead taking points, getting closer to food and scoring, by returning on its team side. The agent is also

given a reward if it kills Pacman, but it is not as great as the other rewards, since eating a Pacman is considered to be more on the defensive side.
On the other hand, our defensive player is motivated to take out the opposing Pacman by actively searching for opponents and patrolling around the team's points. Therefore, it is rewarded for eating enemy Pacman, patrolling its team food, and going towards food that has recently been eaten.

Lastly, weights are continuously saved into a folder for both offensive and defensive players during training so that they may be used in game tournaments right once, enabling the assessment of our team's performance.

2.2.2 Hyper-parameters tuning

Considering the multiple hyper-parameters in approximation Q-learning that need to be adjusted, we ran a number of tests to optimize our approach. In particular, we experimented with different values of epsilon, learning rate, and weightings applied to the behaviors of each agent.

At first, we assigned our agents reward values depending on four actions: food consumed, kill Pacman, scoreDenoted, and deaths. We set these values to [-20, 15, 5, 12] for the offensive agent and [-20, 15, 20, 7] for the defensive agent. As was previously mentioned, the defensive agent should be given priority for Pacman eliminations, while the offensive agent should be more highly rewarded for eating. We ran tests with different epsilon values and learning rates using various reward settings. After putting each agent through 200 episodes of training against a baseline opponent, we put them through 50 episodes of testing against the identical baseline team to evaluate performance.

For our initial experiments, We used epsilon values of [0.2, 0.5, 0.7] and a learning rate of 0.1. The win rates against the baseline team were 74%, 100%, and 100% over three repetitions for each epsilon value. We chose epsilon = 0.5 for further investigation because, notably, we saw a higher average food score (6.54) with epsilon = 0.5 than with epsilon = 0.7 (5.24), even thought both achieved 100% wins. We then trained again with epsilon = 0.5, experimenting with learning rates of 0.6 and 0.001. Both teams defeated the baseline team 100% of the time, but we found that a learning rate of 0.6 was ideal based on the higher average score.

With epsilon = 0.5 and learning rate = 0.6 established as our optimal hyper-parameters, we adjusted the weights assigned to each variable to refine our team's performance. We started with both agents' deaths and scoreDenoted set to -100 and 50, respectively. We kept the initial values for killPacman and foodtaken. We then made a small adjustment to killPacman and foodtaken for both agents, bringing them down to [Defense: -20, 15, 22, 10; Offense: -20, 15, 8, 14]. Both sets of weights produced 100% victory percentages versus the baseline team, but because the original weights had a higher average score (7.5), they were preferred.

We concluded that epsilon = 0.5 and learning rate = 0.6 are the ideal hyper-parameters. The ideal weight configurations for the offensive and defensive agents, respectively, were [-20, 15, 5, 12] and [-20, 15, 20, 7]. We trained over 1000 episodes using these values, achieving the best weights for our features and then conducted tournaments with all created teams, including the baseline, the outcomes of which are discussed later.

2.3 Heuristic Agent

2.3.1 Implementation

This section is about implementing a rule-based heuristic agent. It includes two different classes with different strategies named Offensive Heuristic Agent and Defensive Heuristic Agent to optimize the performance of Pacman. These classes inherit from the Heuristic Agent which with the help of provided CaptureAgent uses heuristic functions to select the best action and evaluate the game by monitoring the successorScore.

Best action: A simple version of MinMAx search algorithm is implemented to find the best action for the states. This algorithm is commonly used in two player games. It aims to maximize Pacman score and minimize the opponent score by considering offensive and defensive strategies.
Game evaluation: The evaluation function is based on a set of features that captures related

features of the environment and contains the score of game states. This function help to measure the performance of each action to select the best one. Various settings for weights were tested during the implementation and the provided values have the best performance.

Features

This section explains the provided features of two agents in details.

Offensive Heuristic Agent Features

- **Successor score**: The goal of using this feature is to help the agent to find the best action, the highest score of the successor state, the best action to be selected.
- **Distance to food**: This feature tries to find the agent position and gets its distance with food pellets and then selects the the nearest food pellet to eat. Because the goal of an offensive agent to eat food as much as it can, this feature helps to select the actions to take Pacman closer to food.
- **Number of invaders**: Because it is important for Pacman to be aware of its opponent to avoid being deleted, this feature is defined by counting the number of opponent Pacman agents.
- **Invader distance**: With the help of previous feature, if there was an opponent Pacman agent in the game, this feature finds the closest one to help the actions that minimize this distance and avoid being eaten.
- **Distance to capsule**: This feature finds the distance between the agent and the capsule. The capsules in the game can help Pacman with temporary invincibility for winning the game.
- **Distance to teammate**: In a team-based game like Pacman, distance to the teammate is a significant feature to help the actions that maximize coordination. For this approach, the feature finds the position of the agents and calculates the distance between them.

Defensive Heuristic Agent Features

- **Number of invaders**: Like the reasons mentioned in offensive agent, the defensive agent should be aware of number of invaders to measure the threat.
- **On defense**: This feature is defined to show whether the agent is a ghost and should protect the food pellets of its own zone and adjusts its strategy based on that.
- **Invader distance**: For a defensive agent, it is important to know the distance to the nearest invader. This helps to measure the threat and prioritize defending actions.
- **Stop and reverse**: As the Pacman can attack and defend the food sources too, these features are added for the agent to penalize stopping and reversing actions to encourage continuous movement.
- **Distance to teammate**: Collaboration with teammate is also important for defensive agent to help prioritizing actions that facilitate defensive coordination.
- **Distance to food**: As a defender should prevent opponents from reaching food pellets, this feature tries to evaluates the distance to the nearest food pellet being defended to take the best action.

3 Results

To assess the performance of various agent implementations, a tournament was held involving 50 games played between each pair of agents. Table 1 presents the win percentages for each matchup, where the row agent is denoted in red and the column agent in blue.

The Q-Learning agent emerged as the most outstanding performer overall, winning 94% of games against the baseline agent, 100% against the heuristic agent, and 76% against the MCTS agent.

This underscores the effectiveness of the reinforcement learning methodology in learning an optimal policy through repeated gameplay experience.

The heuristic agent also delivered a strong display, winning 78% of the games against the baseline agent. However, it struggled against the more sophisticated Q-Learning and MCTS agents, winning only 0% and 2% of the games, respectively. This suggests that, while handcrafted heuristics can outperform basic strategies, they may be limited to more advanced algorithms.

The MCTS agent exhibited mixed performance, winning 62% against the baseline, but only 34% and 36% against the Q-Learning and heuristic agents, respectively. This might indicate room for further enhancement of the MCTS implementation or parameter tuning.

Interestingly, the baseline agent managed to win 36% of thr games against the MCTS agent, indicating that even a simple strategy can occasionally surpass more complex algorithms.

Table 2 presents the percentage of draws for each match-up. The Q-Learning agent had the lowest overall draw rate, with 0% draws against the baseline and MCTS agents. This can be attributed to the learned policy's ability to consistently reach a winning or losing state.

The heuristic and baseline agents had relatively high draw rates against each other at 54% and 66%, respectively, indicating that their strategies may have led to more stalemate situations.

Another interesting observation we can make is that Q-Learning vs Q-Learning team, while having many draws as someone could expect, it also has 38% wins while been red, while 0% while been blue. This is probably because of the fact that it was trained as red and even though it achieved great results also as blue, it is still a bit better as red.

The MCTS team has the smallest percentage of draws against itself, and has balance between one team winning the other. This is also the case with the others teams , which additionally have many draws without one outperforming the other while against their selves.

The outcomes of the experiments underscore the advantages and limitations of various agent implementations, suggesting that the Q-Learning agent proved to be the most efficient in the Pacman Capture the Flag environment.

Agent	**Baseline**	**Heuristics**	**Q-Learning**	**MCTS**
Baseline	12%	14%	36%	12%
Heuristics	78%	20%	0%	2%
Q-Learning	**94**%	**100**%	**38**%	**76**%
MCTS	62%	70%	34%	36%

Table 1: Win percentages for a 50-game Pacman CTF tournament between different agent teams. Each cell shows the percentage of games won by the row team (red) against the column team (blue). Percentages may not sum to 100% due to ties.

Agent	**Baseline**	**Heuristics**	**Q-Learning**	**MCTS**
Baseline	66%	6%	0%	22%
Heuristics	6%	54%	14%	40%
Q-Learning	0%	0%	62%	0%
MCTS	26%	18%	36%	14%

Table 2: Percentages of draws for a 50-game Pacman CTF tournament between different agent teams. Each cell shows the percentage of draws by the row team (red) against the column team (blue).

4 Future Work

In this project, our MCTS agent is far from perfect, and the results show that. One way to improve the overall performance is by employing several techniques mentioned in [5]. Specifically, incorporating techniques such as early playout terminations, implicit minimax backups, quality-based rewards, and progressive bias - could potentially enhance the strength and decision-making capabilities of our agent. Moreover, the findings from [5] motivate the investigation of other state-of-the-art MCTS enhancements, such as rapid action value estimation (RAVE), progressive history, and adaptive playout policies. Integrating and benchmarking these techniques within our Pacman MCTS agent could potentially unlock further performance improvements.

Moreover, even with the very good performance of the Q aproximation team, it can probably become even better in the future by introducing more features and weights, like information regarding scared ghosts and power capsules.

5 Conclusion

It is clear from the thorough analysis of numerous agent implementations in the Pacman Capture the Flag (CTF) environment that each strategy differs greatly in terms of efficacy. The Q-Learning agent is the best performance, demonstrating its proficiency in using reinforcement learning approaches to develop optimal policies. Its strong performance versus both baseline and heuristic-based opponents highlights its adaptability and capacity to thrive in dynamic, multi-agent environments. On the other hand, the heuristic-based AI performs well versus less complex opponents but poorly against more advanced ones, suggesting that the flexibility of handcrafted heuristics is limited. The Monte Carlo Tree Search (MCTS) agent performs erratically, indicating potential for enhancement via more implementation refinement or parameter adjustment. Overall, these findings underscore the importance of hyperparameter optimization and the potential for continued research to enhance agent performance in complex gaming environments like Pacman CTF.

References

[1] Berkeley ai contest. http://ai.berkeley.edu/contest.html. Accessed: March 28, 2024.

[2] L. Buşoniu, D. Ernst, B. De Schutter, and R. Babuška. Approximate reinforcement learning: An overview. Technical report, Proceedings of the 2011 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL 2011), Paris, France, Apr. 2011.

[3] Gang Chen, Weihua Cao, Xin Chen, and Min Wu. Multi-agent q-learning with joint state value approximation. In *Proceedings of the 30th Chinese Control Conference*, pages 4878–4882, 2011.

[4] Gibberblot. Function approximation, n.d. Accessed: March 28, 2024.

[5] Chu-Hsuan Hsueh, I-Chen Wu, Wen-Jie Tseng, Shi-Jim Yen, and Jr-Chang Chen. An analysis for strength improvement of an mcts-based program playing chinese dark chess. *Theoretical Computer Science*, 644:63–75, 2016. Recent Advances in Computer Games.